VIETNAM NATIONAL UNIVERSITY, HANOI — UNIVERSITY OF SCIENCE
Course: Computer Security

Hands-on Report

Cross-Site Request Forgery (CSRF) Attack

TOPIC

CSRF Attack on the Elgg Social Platform via the Fetch API

Replacing XMLHttpRequest / Form submit with `fetch()` + `URLSearchParams` / `FormData`

Team	Group 6 (03 members)	
Nguyen Trung Thanh	ID 23001935	— Theory, HTTP analysis, defenses
Nguyen Cong Tri	ID 23001939	— Elgg, Docker, Fetch API, testing
Nguyen Minh Thao	ID 23001936	— Evidence, slides, video
Instructor	_____	
Date	November 13, 2026	

Ethics note: All content serves educational purposes only, inside a local lab environment (Docker) owned by the team — it is not aimed at any individual or organization.

Abstract

This report presents the study and hands-on experiment of a **Cross-Site Request Forgery (CSRF)** attack against the open-source social-networking application **Elgg**. The improvement over the original lab is to fully replace `XMLHttpRequest` and the `form submit` mechanism with the modern **Fetch API** combined with `URLSearchParams` or `FormData`. The exploitation goal is to change the victim's *About me* profile field to the string `CSRF attack by <ID>`.

Unlike many reports that stop at description, the team actually **built and ran** a local Docker lab (Elgg **3.3.3**, loopback-isolated), captured **real screenshots** from the Chrome browser and verified results at both the UI and the HTTP layers. The report analyses the HTTP packet, explains the core mechanisms (`credentials: 'include'`, Simple Request, `mode: 'no-cors'`), illustrates the attack-flow diagram, reports a **technical finding** about how Elgg stores a profile field (annotation vs. metadata), and — most importantly — **proves experimentally** the effectiveness of two independent defense layers: the *anti-CSRF token* and the *SameSite cookie*. The report ends with answers to 15 theory questions, a reproduction procedure and an AI-usage disclosure.

Contents

1 Introduction and objectives **2**
1.1 Background . . . 2
1.2 Project objectives . . . 2

2 CSRF theoretical background **2**
2.1 Three conditions of an attack . . . 2
2.2 Cookies and implicit trust . . . 2
2.3 Same-Origin, Same-Site and cross-site . . . 3
2.4 The "can send but cannot read" gap . . . 3
2.5 CSRF versus XSS . . . 3
2.6 Threat model . . . 3

3 Two experimental environment models **3**
3.1 The standard SEED Labs model . . . 3
3.2 The team's local Docker lab (actually run) . . . 4
3.3 Comparing the two models . . . 4

4 HTTP request analysis with DevTools **4**
4.1 Capturing the request with the Network tab . . . 5

5 Exploitation with the Fetch API **5**
5.1 Comparing the three methods . . . 5
5.2 Two key mechanisms . . . 5
5.3 Exploit code (URLSearchParams variant) . . . 6
5.4 Exploit code (FormData variant) . . . 6

6 Attack-flow diagram **7**

7 Experimental results on the real lab **7**
7.1 Test-case matrix . . . 7
7.2 Visual evidence . . . 7
7.3 HTTP-layer verification (curl) . . . 8
7.4 Detailed per-case narrative . . . 8

8 Technical finding: annotation vs. metadata **9**

9 Verification and defenses **9**
9.1 Re-enabling Elgg's anti-CSRF token . . . 9
9.2 The independent role of the SameSite cookie . . . 10
9.3 Defense-in-Depth . . . 10
9.3.1 How each layer works . . . 10

10 Automated testing and reproducibility **10**

11 Task allocation **11**

12 AI-usage disclosure **11**

13 Discussion, limitations and future work **11**
13.1 What was achieved . . . 11
13.2 Limitations . . . 12
13.3 Future work . . . 12

14 Conclusion **12**

A Appendix A: Answers to the 15 questions (summary) **12**

B Appendix B: Reproduction details **13**

1. Introduction and objectives

1.1 Background

CSRF is a vulnerability at the web application logic layer that exploits the browser's *implicit trust* mechanism: when a request is sent to a domain, the browser automatically attaches all cookies belonging to that domain — including the authenticated session cookie. The attacker abuses this to make the victim's browser (while logged in) issue a state-changing request that the victim never intended. The dangerous part is that the attacker *does not need the password* nor to steal the cookie: it merely *borrows* the victim's existing session at the moment the victim visits a malicious page.

1.2 Project objectives

- **Target system:** the Elgg application running inside a Docker container; the simulated victim account is *Alice*.
- **Exploitation behavior:** from an attacker's web page (different origin), forge an HTTP POST to the endpoint `/action/profile/edit` to change the victim's *About me* field to `CSRF attack by 23001939`.
- **Mandatory technical constraints:**
  1. ✘ Do not use `XMLHttpRequest`.
  2. ✘ Do not use a self-submitting `<form>` (causes a redirect, leaves traces).
  3. ✔ Must use the **Fetch API** + `URLSearchParams` or `FormData`.
- **Evidence requirement:** change About me and simultaneously *prove* why the attack succeeds without defenses and fails with defenses enabled.

> All operations are performed only in a local lab owned by the team, isolated on `localhost`. Applying these techniques to any unauthorized system violates the law and academic regulations. The sole purpose is to *learn attack in order to learn defense*.

2. CSRF theoretical background

2.1 Three conditions of an attack

1. The victim has a **valid active session** (session cookie) with the target application.
2. The target action **changes state** (edit profile, transfer money, add friend…) and has **predictable parameters**.
3. The application **lacks defenses** (no SameSite Strict/Lax, no anti-CSRF token, no `Origin` check).

If any condition is missing, the attack fails — which is precisely the basis for designing defenses (Section 9).

2.2 Cookies and implicit trust

When Alice logs in, Elgg issues a session cookie (e.g. `Elgg=p8nm1rqhk8…`) stored in the browser and bound to Elgg's host. The browser's **mechanical rule** is: *every* request sent *to* that host will *automatically* carry the cookie, regardless of which page issued it. The browser decides to attach a cookie based on the request's *destination*, not on the *origin* of the page that emitted it. This "indifference" is what creates the CSRF vulnerability.

2.3 Same-Origin, Same-Site and cross-site

Concept	Definition	Example in the lab
Same-origin	Same *scheme* + *host* + *port*	`elgg.csrf.localhost:8080` with itself
Same-site	Same *site* (eTLD+1), possibly different subdomain/port	`attacker.csrf.localhost` & `elgg.csrf.localhost` (same site `csrf.localhost`)
Cross-site	Different site (different eTLD+1)	`127.0.0.1` vs. `elgg.csrf.localhost`

The distinction between *same-site* and *cross-site* is the key to understanding the `SameSite` cookie: this attribute blocks the cookie on *cross-site* requests but still allows it on *same-site* requests. The team deliberately deploys two attacker pages — one *same-site* and one *cross-site* — to isolate the role of SameSite (see Section 7).

2.4 The "can send but cannot read" gap

The Same-Origin Policy (SOP) forbids a page's JavaScript from *reading* the content/response of a different-origin page — but it does **not** forbid *sending* a request. This gap is the foundation of CSRF: the attacker need not read the result, it only needs the state-changing action to be executed on the server side. This is also why `mode: 'no-cors'` (an *opaque* response that JS cannot read) still suffices for a successful attack.

2.5 CSRF versus XSS

Avoid confusion: **XSS** injects and runs code within the victim page's context (can read cookies, tokens…), whereas **CSRF** *cannot* read anything of the target page — it merely *abuses* the browser auto-attaching the cookie. Hence an anti-CSRF token defeats CSRF (the attacker cannot read the token) but does *not* defeat XSS (XSS code runs inside the page and can read the token).

2.6 Threat model

Element	Description in the lab
Asset to protect	Integrity of the user profile (the About me field)
Attacker	Owner of a malicious website; *no* access to the Elgg server, *does not* know Alice's password
Attacker capability	Can make Alice's browser emit a request; *cannot* read the response (SOP)
Trigger condition	Alice is logged in and visits the attacker page
Attacker goal	Execute a state-changing action on Alice's behalf without her intent

This model shows CSRF as an *implicit privilege-abuse* flaw: the attacker borrows the *authority* of the session rather than stealing credentials. Consequently, the safe-design principle is to *never* use `GET` for state-changing actions, and to always bind an action to a proof of "genuine user intent" (token/Origin/SameSite).

3. Two experimental environment models

3.1 The standard SEED Labs model

The original SEED Labs CSRF exercise uses three containers with virtual hostnames mapped via `/etc/hosts`:

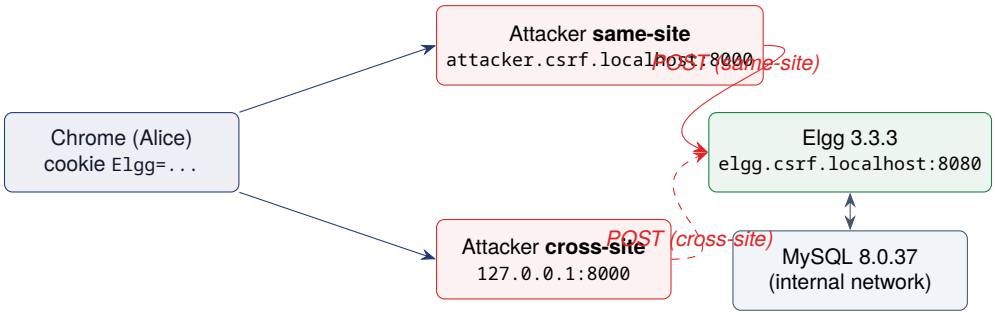
Service	Address	Role
seed-server	`www.seed-server.com` (10.9.0.5)	Apache + the Elgg application (target)
mysql	10.9.0.6	Elgg user database
attacker32	`www.attacker32.com` (10.9.0.7)	The attacker's web server

This model is the standard *conceptual framework* for presenting the theory; however both hosts are distinct `*.com` (therefore cross-site in practice) and it relies on the token already being disabled in the SEED image.

3.2 The team's local Docker lab (actually run)

To precisely control the experimental conditions and gather real evidence, the team built its own lab, fully isolated on loopback:

Component	URL	Role / condition
Elgg (victim)	`elgg.csrf.localhost:8080`	Elgg **3.3.3**, host-only cookie, HTTP loopback
Attacker same-site	`attacker.csrf.localhost:8000`	Different origin, *same site* `csrf.localhost`
Attacker cross-site	`127.0.0.1:8000`	*Different site* from Elgg
MySQL	(in the Docker network)	8.0.37, no published port



Lab architecture: Alice's browser, two attacker pages (same-site / cross-site), Elgg and MySQL.

The application version is verified at runtime with `labctl version`: Elgg `3.3.3`, image digest `sha256:5d8a25b1…708f7`
The lab has **two modes** toggled via the `labctl` tool:

- `protected` (default): Elgg checks `__elgg_token`/`__elgg_ts` as usual ⇒ the attack *must fail*.
- `vulnerable`: disables the token check *only for* the `/action/profile/edit` action ⇒ demonstrates the consequence of losing a defense layer.

Why two attacker hosts?

Placing one *same-site* and one *cross-site* attacker lets us change **exactly one variable** at a time: keep the `vulnerable` mode (token disabled) and only switch *same-site* ↔ *cross-site*. If the outcome differs, the cause *can only be* the `SameSite` cookie. This is controlled-variable methodology to attribute cause rigorously.

3.3 Comparing the two models

The standard SEED model is used to explain concepts and cross-check documentation; the local Docker lab is used to *actually run, gather evidence and control variables*. The two models complement each other: the concepts drawn from SEED are *verified* on the team's real lab. The results (Section 7) and defense (Section 9) sections all rely on the Docker lab that was run.

4. HTTP request analysis with DevTools

To forge accurately, we must first capture a legitimate request. Log in as Alice (`alice`/`seedalice`), open *Edit Profile*, open `F12` → `Network` (enable *Preserve log*), edit the About me field and click **Save**.

Listing 1: POST request when saving the profile (abridged, from the real lab)

```
POST /action/profile/edit HTTP/1.1
Host: elgg.csrf.localhost:8080
Content-Type: application/x-www-form-urlencoded
Cookie: Elgg=p8nm1rqhk8igl0ivf1i5qkdl3m; ...

```

```
name=Alice&description=<p>...</p>&accesslevel[description]=2
&guid=56&__elgg_ts=1794550779&__elgg_token=RG40vT3Fxs...
```

Parameter	Meaning	Value (lab)
`guid`	Victim identifier (Alice)	`56`
`name`	Display name	`Alice`
`description`	**About me content**	`CSRF attack by 23001939`
`accesslevel[description]`	Visibility (2 = Public)	`2`
`__elgg_token / __elgg_ts`	Anti-CSRF token + timestamp	*(secret — NEVER move to attacker)*

4.1 Capturing the request with the Network tab

1. Open `F12` → `Network`, enable *Preserve log* so requests survive a redirect.
2. Perform a legitimate action (Save the profile); find the `POST` request to `/action/profile/edit`.
3. Inspect the *Payload/Request* tab to read the fields and the *Headers* tab to confirm `Content-Type`, `Cookie`.
4. Record the *endpoint* and the *public fields*; confirm the existence of `__elgg_token` but **do not** copy its value.

Copy only the *public fields* (`guid`, `name`, `description`, `accesslevel`) to the attacker page. **Never** enter `__elgg_token`, `__elgg_ts`, a cookie or a password into the attacker — doing so is "handing yourself the key" and destroys the meaning of the experiment. The team's attacker tool actively *rejects* any field containing `token`/`cookie`/`password`.

5. Exploitation with the Fetch API

5.1 Comparing the three methods

Criterion	Form submit	XHR	Fetch API
Stealth (no page change)	✘	✔	✔
Modern syntax (Promise/async)	✘	✘	✔
Control over `mode`/`credentials`	limited	yes	✔ explicit
Compliant with the assignment	✘ banned	✘ banned	✔ required

5.2 Two key mechanisms

`credentials: 'include'` — send cookies cross-origin

By default, a cross-origin `fetch()` uses `credentials: 'same-origin'` and therefore does *not* send Elgg's cookie. One must set `credentials: 'include'` so the browser auto-attaches the victim's session cookie — without it, the server treats the request as an anonymous visitor and the attack fails. Note: this flag is only a *request*; the browser may still refuse to attach the cookie if it carries a `SameSite` attribute and the request is cross-site.

Simple Request + `mode: 'no-cors'` — bypass Preflight

A `Content-Type` of `application/x-www-form-urlencoded` (URLSearchParams) or `multipart/form-data` (FormData) are both *CORS-safelisted* ⇒ this is a **Simple Request**, with no Preflight `OPTIONS`. Even though SOP blocks JavaScript from *reading* the response (opaque), the Elgg server *still* receives the valid cookie and overwrites the database successfully.

5.3 Exploit code (URLSearchParams variant)

Listing 2: Attack core using the Fetch API + URLSearchParams

```
const payload = new URLSearchParams();
payload.append('name', 'Alice');
payload.append('description', 'CSRF attack by 23001939');
payload.append('accesslevel[description]', '2');   // 2 = Public
payload.append('guid', '56');                       // Alice's GUID

fetch('http://elgg.csrf.localhost:8080/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',          // Simple Request -> no Preflight OPTIONS
  credentials: 'include',   // REQUIRED: attach victim's session cookie
  body: payload             // URLSearchParams sets a safe Content-Type
});
```

5.4 Exploit code (FormData variant)

Listing 3: Fetch API + FormData variant

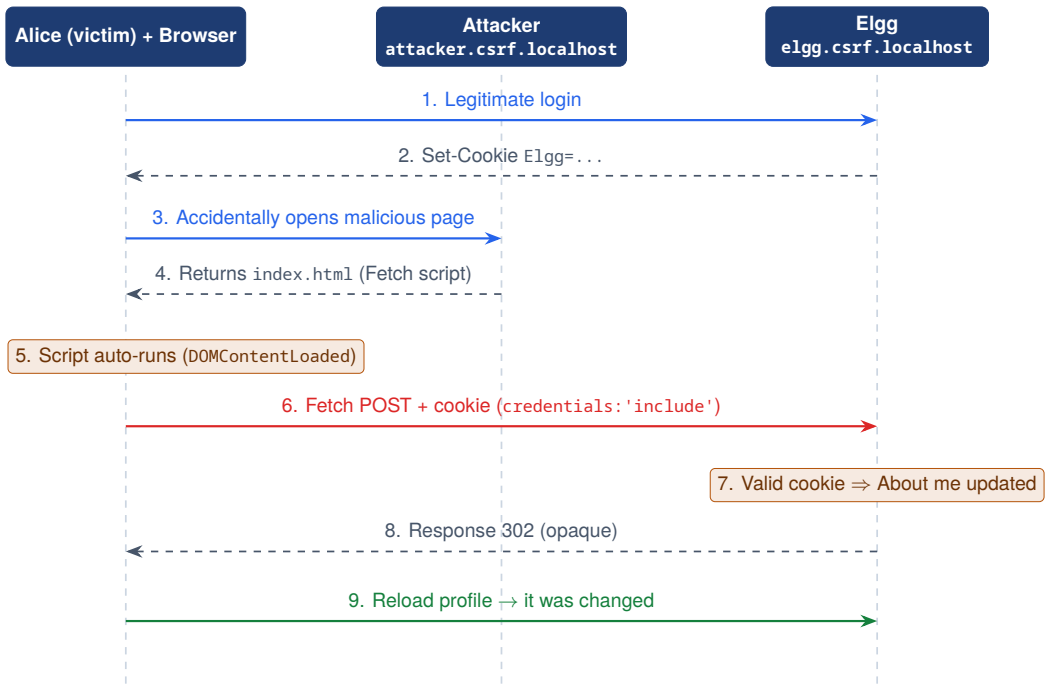
```
const payload = new FormData();
payload.append('name', 'Alice');
payload.append('description', 'CSRF attack by 23001939');
payload.append('accesslevel[description]', '2');
payload.append('guid', '56');

fetch('http://elgg.csrf.localhost:8080/action/profile/edit', {
  method: 'POST',
  mode: 'no-cors',
  credentials: 'include',
  body: payload            // DO NOT set Content-Type: browser generates the boundary
});
```

URLSearchParams or FormData?

Both create a *Simple Request* so neither triggers Preflight. `URLSearchParams` produces `application/x-www-form-urlencoded` (compact, good for simple forms); `FormData` produces `multipart/form-data` with a `boundary` (good when attaching files). The shared key point: **do not** set the `Content-Type` header manually so the browser keeps a *safelisted* value. The team provides both variants in the source code.

6. Attack-flow diagram



Sequence: (1–2) Alice logs in, receives a cookie. (3–5) Alice accidentally opens the attacker page, the Fetch script auto-runs. (6) Fetch POST carries the cookie (thanks to `credentials:'include'`). (7) If defenses are missing, Elgg updates About me. (8–9) The response is opaque; when Alice reloads her profile she sees it was changed.

7. Experimental results on the real lab

The team ran the full scenario on a real Chrome browser (driven by automation) and checked every step. Verified Elgg version: **3.3.3**.

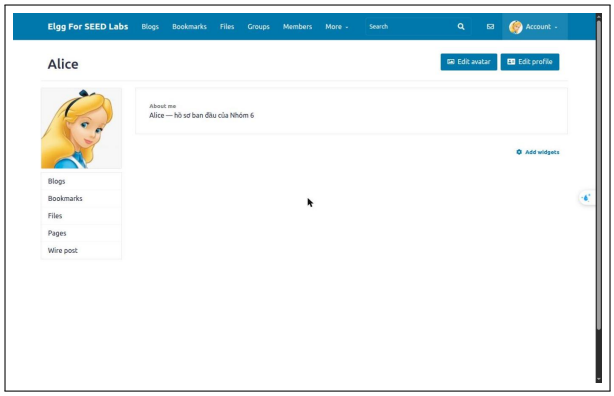
7.1 Test-case matrix

TC	Mode	Attacker	Same-site?	Expected	Res.
TC-B01	protected	— (Elgg UI)	—	Legit Save; request has token	✔
TC-B04	protected	same-site	yes	Blocked, About me unchanged	✔
TC-B03	vulnerable	`127.0.0.1`	**no**	Blocked by SameSite	✔
TC-B05/06	vulnerable	same-site	yes	**Success**, About me changed	✔

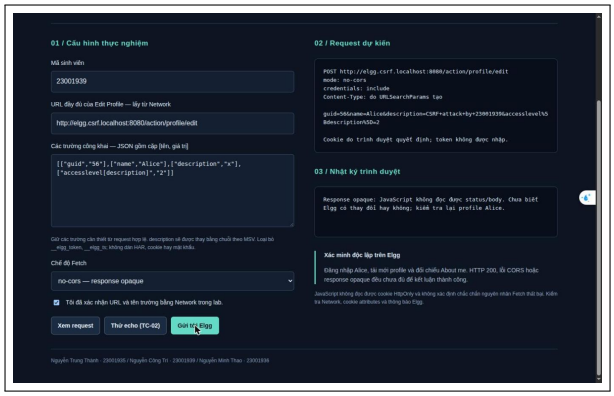
Only *one* variable changes between TC-B03 and TC-B05/06 (cross-site ↔ same-site). The opposite outcomes (blocked ↔ success) are precisely the proof of the `SameSite` cookie's role.

7.2 Visual evidence

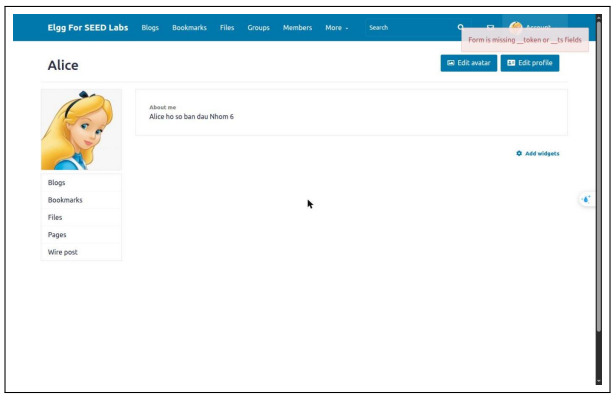
In `vulnerable` mode with a same-site attacker, the About me field is changed to `CSRF attack by 23001939` with no warning or redirect — a fully successful exploit. Conversely, in `protected` mode (token on) the same payload is rejected by Elgg with a missing-token message; and in `vulnerable` but cross-site, the SameSite cookie strips the session so the profile is not changed either.



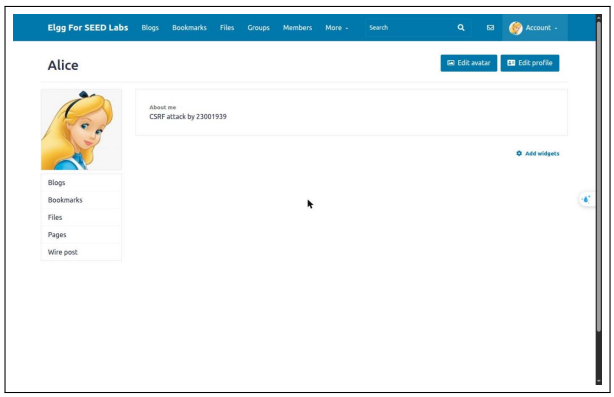
(a) **Before:** Alice's baseline About me.



(b) Attacker page: the planned Fetch payload, *without* a token.



(c) **TC-B04 (protected):** Elgg reports *"Form is missing __token or __ts fields"*; About me unchanged.



(d) **TC-B06 (vulnerable, same-site):** About me = `CSRF attack by 23001939`.

7.3 HTTP-layer verification (curl)

To isolate *only* the token effect (curl always attaches the cookie regardless of origin, so it does not exercise SameSite), the team repeated the *tokenless* POST with `curl`:

Mode	Tokenless POST	HTTP redirect	About me changed?
protected	blocked	→ `/` (home)	✘
vulnerable	accepted	→ `/profile/alice`	✔
restored	—	—	✘

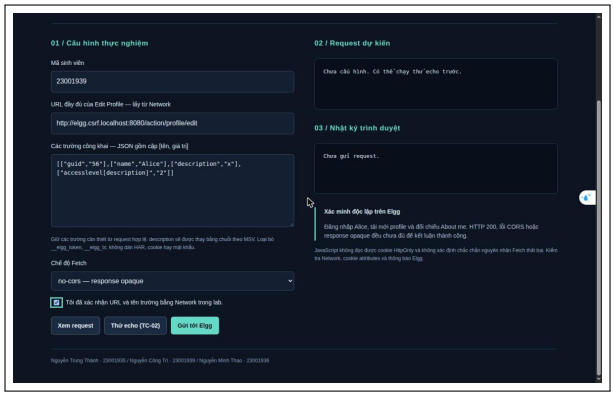
The difference in the *redirect target* (home when blocked, the profile page when successful) is a clear corroborating signal, consistent with the UI observation.

7.4 Detailed per-case narrative

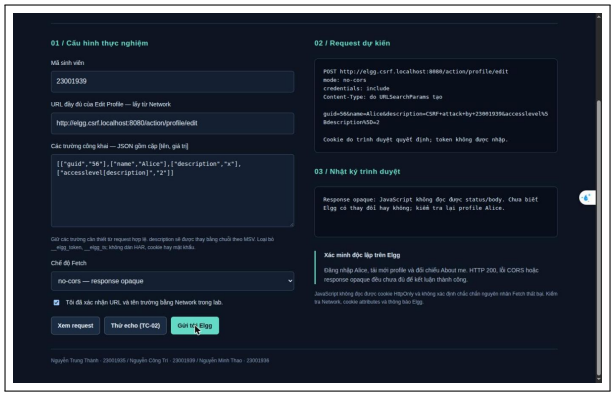
TC-B01 — Login & legitimate Save.
Log in as Alice, open Edit Profile, set the baseline About me, click Save. The `POST /action/profile/edit` request carries a full `__elgg_token`/`__elgg_ts`; the profile updates correctly. This is the template to mimic and the standard recovery path.

TC-B04 — Protected blocks the attack.
Keep `protected` mode. The same-site attacker page sends a POST (`credentials:'include'`, `no-cors`). The browser *does* attach the cookie (same-site) but the payload *lacks* the token. Elgg shows the red message *"Form is missing __token or __ts fields"*; About me stays at baseline.

TC-B03 — SameSite blocks cross-site.
Switch to `vulnerable` (token off). Send from `127.0.0.1` (cross-site). The `SameSite` cookie makes



(e) Attacker UI: configuring the endpoint + public fields (no token).



(f) **TC-B03:** attack from `127.0.0.1` (cross-site) in vulnerable mode — the profile is *still* unchanged.

the browser *not* attach the session on a cross-site POST; Elgg treats it as a stranger so the profile is unchanged — proving SameSite protects *independently* of the token.

TC-B05/06 — Successful attack.
Still `vulnerable`, send from the same-site attacker. SameSite allows the same-site cookie and the token is off ⇒ Elgg accepts it. Reload the profile: About me = `CSRF attack by 23001939`. This is the only valid evidence (direct comparison of the displayed field).

8. Technical finding: annotation vs. metadata

While restoring the profile, the team discovered an important Elgg implementation detail:

- The web action `/action/profile/edit` stores the About me field as an **annotation** `profile:description` (table `elgg_annotations`) — and that is the value the *profile page displays*.
- Meanwhile, the PHP property `$user->description` maps to **metadata** (table `elgg_metadata`).

Consequence: the `reset-profile` tool initially reset only the *metadata*, so although it reported "success" the page still displayed the attack string; clearing the cache (`elgg-cli flush`), restarting the container or deleting the Stash cache did *not* help, because the data lives in the *annotation*. The correct recovery is to re-edit About me *through the Edit Profile UI* (going through the proper action, overwriting the annotation), or to directly update the `profile:description` annotation.

Lesson

When verifying a result, one must inspect the *data source the UI actually displays*. A "successful" change at the DB layer but in the wrong table still yields a wrong displayed result. This finding also restates that a CORS error or an opaque response is *not* evidence — only a direct comparison of the displayed field is valid evidence.

9. Verification and defenses

9.1 Re-enabling Elgg's anti-CSRF token

After restoring the `__elgg_token`/`__elgg_ts` check, re-run the exact same payload:

	Defense OFF	Defense ON
Payload has `__elgg_token`?	No	Still no
Server response	DB updated (302)	**Rejected** (missing token)
Alice's About me	Changed	**Unchanged**

Because SOP forbids the attacker page's JavaScript from reading the token on the Elgg page, the forged request always lacks a valid token ⇒ it is blocked. This is a vivid demonstration of the *Synchronizer Token Pattern*.

9.2 The independent role of the SameSite cookie

Experiment TC-B03 shows: *even when* the token is disabled (`vulnerable`), a POST from the **cross-site** attacker (`127.0.0.1`) *still* cannot change the profile — because the browser does not attach the session cookie to a cross-site request (the `SameSite` cookie). Elgg treats the request as a stranger and does not act on Alice's behalf. Thus **each layer** (token or SameSite), on its own, suffices to block this attack.

9.3 Defense-in-Depth

Layer	Measure	What it blocks
Cookie	`SameSite=Lax/Strict`	Browser does not send the cookie cross-site
Application	Anti-CSRF token (Synchronizer)	Attacker cannot read the token (thanks to SOP)
Application	Double Submit Cookie	Attacker cannot read the cookie to place in the body
Header	`Origin` / `Sec-Fetch-Site` check	Reject state-changing requests from unknown sources
UI	`X-Frame-Options` / CSP `frame-ancestors`	Prevent Clickjacking via iframe

9.3.1 How each layer works

- **SameSite cookie:** enforced by the *browser*. `Strict` blocks the cookie for any request originating from another site; `Lax` (today's default) still sends the cookie for top-level navigation (typing a link, clicking) but blocks it for background cross-site POSTs — just enough to stop CSRF without breaking UX. *Limitation:* does not protect against a *same-site* attack (e.g. a hijacked subdomain).
- **Anti-CSRF token (Synchronizer):** enforced by the *server*. The server embeds a per-session random token into each form; a valid request must return the correct token. Thanks to SOP, a different-origin attacker cannot read the token. *Limitation:* requires token lifecycle management; does not stop XSS.
- **Double Submit Cookie:** the server sends the token via both a cookie and a hidden field, then compares them — no server-side state needed. *Limitation:* weak if the attacker can write the cookie (e.g. via a subdomain).
- **`Origin`/`Sec-Fetch-Site` check:** the server rejects state-changing requests from unknown sources. Simple, effective for modern APIs. *Limitation:* some proxies/configurations may strip the header.

Practical recommendation

Combine `SameSite=Lax` (the default layer) + an anti-CSRF token (the application layer) + an `Origin` check for sensitive endpoints. Never use `GET` for a state-changing action. The *defense-in-depth* principle: if one layer fails, the others still block the attack.

10. Automated testing and reproducibility

Beyond the manual experiment, the team maintains an automated test suite to ensure the payload is well-formed and the environment is healthy:

Test suite	Content	Result
`test_request_format.mjs` (Node)	6 tests building the Fetch payload; reject token/cookie input	✔ 6/6
`test_environment.py`	4 tests of the attacker server: echo, static allowlist, no cookie leak	✔ 4/4
`test_security_controls.py` (live)	services healthy; Elgg has no CORS allowance	✔ 2/2
`docker compose config`	compose syntax check	✔

Listing 4: Lab reproduction procedure (summary)

```
python3 lab/scripts/labctl.py up          # bring up Elgg + MySQL + attacker
python3 lab/scripts/labctl.py version     # verify Elgg 3.3.3
# ... log in as Alice, capture request, configure attacker, try sending ...
python3 lab/scripts/labctl.py mode vulnerable   # disable token (deliberate)
python3 lab/scripts/labctl.py mode protected    # re-enable token
python3 lab/scripts/labctl.py reset-profile     # restore About me
```

11. Task allocation

Member	ID	Focus
Nguyen Trung Thanh	23001935	CSRF theory, HTTP request analysis, defenses
Nguyen Cong Tri	23001939	Elgg, Docker, Fetch API code, testing
Nguyen Minh Thao	23001936	Evidence, slides, video

Mandatory common baseline (100% of members): each member runs Docker themselves, captures the request themselves and changes About me to their own student ID; all take part in the cross-examination of 15 questions.

12. AI-usage disclosure

1. **Level 1 — Critique:** used AI to discuss `credentials: 'include'`, Simple Request and CORS; students wrote the code themselves.
2. **Level 2 — Co-creation:** students designed the report/slide/diagram structure, AI helped polish wording and LaTeX formatting.
3. **Level 3 — Prompted generation:** AI helped scaffold the `labctl`/attacker tooling and drive the test automation; students configured the real values, ran them and own the results.

All experimental results and their accuracy are the direct responsibility of the team.

13. Discussion, limitations and future work

13.1 What was achieved

- A real end-to-end run on the Chrome browser with Elgg 3.3.3; visual evidence of all three states (before / blocked / success).
- Rigorous variable control to *isolate* the roles of the anti-CSRF token and the SameSite cookie.
- Strict adherence to ethics: no token/cookie entered into the attacker; the tool actively rejects secret fields.

13.2 Limitations

- The standard SEED model (`www.seed-server.com`) is presented only conceptually; all evidence comes from the team's local Docker lab.
- Disabling the token is a *deliberate* lab modification for illustration, not a conclusion that the current Elgg release is flawed.
- A `no-cors` response is opaque so the status code cannot be read from JS; conclusions rely solely on direct profile comparison (handled correctly).

13.3 Future work

- Add an *add friend* attack scenario (only the victim's own guid needed) to compare with the profile-edit scenario.
- Experiment with different `SameSite` values (`Strict`/`Lax`/`None`) and directly observe cookie attach/strip in the Network tab.
- Fix the `reset-profile` tool to reset the correct `profile:description` annotation.

14. Conclusion

The team achieved the objective: changed About me to `CSRF attack by 23001939` via the Fetch API (no XHR/Form), fully analysed the exploitation mechanism, *actually ran* it on the Docker lab (Elgg 3.3.3) with visual evidence, and *experimentally proved* the effectiveness of two independent defense layers (anti-CSRF token and SameSite cookie). The core lesson: CSRF is not prevented by HTTPS; the fundamental remedy is the `SameSite` cookie combined with an anti-CSRF token and request-origin checking. The annotation/metadata finding also stresses a principle: conclude only from directly compared evidence, never from indirect signals.

A. Appendix A: Answers to the 15 questions (summary)

1. **SameSite stops CSRF:** instructs the browser not to send the cookie on cross-site requests (Strict/Lax) ⇒ the forged request loses its authenticated session.
2. **Secret token:** the server embeds a per-session random token into the form; thanks to SOP the attacker cannot read it ⇒ a forged tokenless request is rejected.
3. **HTTPS still worries about CSRF:** HTTPS only secures transport; the cookie is still attached inside the TLS tunnel, so CSRF (an application-layer flaw) is not prevented.
4. **GET `delete.php?pageid=5`:** use a hidden `<img src=...>` or `fetch(...,{credentials:'include'})` to trigger the forged GET.
5. **POST `delete.php`:** use Fetch + `URLSearchParams(pageid=5)`, running silently with no page change.
6. **Finding Boby's guid:** via the profile URL, avatar `src`, the Add-friend link, or the RSS feed — all public.
7. **Unknown user id:** depends on the server — if the id is derived from the session the attack still works; if a large random id must be in the body, it does not.
8. **Attacking an arbitrary guest:** (1) ✘ editing a profile — needs the victim's guid that SOP forbids reading; (2) ✔ add friend — only needs Alice's own guid, the victim's identity is derived from the session.
9. **Session ID in the data part (Double Submit):** the cookie is auto-sent by the browser, but the data part must be set by the origin page's JS; the attacker cannot read the cookie so cannot fill it correctly ⇒ the server detects the cross-site forgery.
10. **Does the browser know cross-site?** Yes — it knows the source and destination origins, sends `Origin`, `Sec-Fetch-Site`.
11. **Does the server know cross-site?** By default with plain HTTP, no; it can know if it checks `Origin`/`Referer`/`Sec-Fetch-Site`/to

12. **Why not rely on Referer alone:** the Referer may be legitimately dropped (`Referrer-Policy`, HTTPS→HTTP, proxy) and is easy to bypass with loose regex.
13. **Why we need to know cross-site:** to allow a state-changing action only when the user genuinely intends it, avoiding borrowed authority.
14. **Ban cross-site cookies entirely?** Technically possible but it breaks navigation, SSO/OAuth and embedded widgets; the solution is `SameSite=Lax` by default.
15. **Facebook iframe:** a request from within an iframe to Facebook is same-site with respect to Facebook (not cross-site); the risk is Clickjacking; mitigate with `X-Frame-Options` / CSP `frame-ancestors`.

B. Appendix B: Reproduction details

- **Versions:** Elgg 3.3.3 (digest `sha256:5d8a25b1…708f7f5`); MySQL 8.0.37; attacker Python 3.12.
- **Public SEED accounts (lab-only):** Alice `alice/seedalice`; admin `admin/seedelgg`.
- **Target string:** `CSRF attack by 23001939` (change per student ID in `.env` or the attacker UI).
- **Recovery:** `labctl mode protected` + re-edit About me to baseline through the Edit Profile UI.

References

[1] W. Du, *Computer & Internet Security: A Hands-on Approach*, 3rd ed.

[2] WHATWG, *Fetch Living Standard*, https://fetch.spec.whatwg.org/.

[3] OWASP, *Cross-Site Request Forgery Prevention Cheat Sheet*.

[4] IETF, *RFC 6265bis — Cookies: SameSite attribute*.

[5] Elgg, *Actions & Security*, https://learn.elgg.org/.